



SKALA

생성형 AI 서비스 개발의 이해 활용 (LangChain)

2026.09

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



1. LLM 기반 AI 서비스 개발
2. Langchain 컴포넌트
3. Integration
4. Runnable의 이해
5. AI 서비스 개발을 위한 UI

LLM 기반 AI 서비스 개발

- 기존 IT 서비스 개발과의 핵심 차별점
- LLM 기반 AI 서비스 개발의 주요 어려움
- LLM 기반 AI 서비스 개발 방법론

기존 IT 서비스 개발과의 핵심 차별점

비교 항목	기존 IT 서비스 개발 (전통적 SW)	LLM 기반 AI 서비스 개발
시스템 성격	결정론적(Deterministic): 동일한 입력에 언제나 동일한 출력 반환	확률론적(Probabilistic): 동일한 입력에도 가변적 결과 생성 가능
핵심 제어 로직	명시적 코드(if/else, 알고리즘, 비즈니스 로직 함수)	자연어 프롬프트, 컨텍스트 엔지니어링, 모델 가중치
입출력 데이터 형태	구조화된 데이터(JSON, SQL 테이블, 원시 타입)	비정형 자연어 텍스트, 멀티모달 데이터
테스트 및 검증	단위 테스트(Unit Test)를 통한 참/거짓 및 상태 검증	벤치마크 평가, LLM-as-a-Judge, 의미론적 유사도 검증

LLM 기반 AI 서비스 개발의 주요 어려움

비결정론적 특성과 출력 제어의 한계

- 환각(Hallucination)
 - 모델이 사실과 다른 내용을 그럴듯하게 생성하는 현상은 완전히 제거하기 어려움
 - RAG 파이프라인의 검색 정확도(Retrieval Accuracy)가 높더라도 파싱 및 추론 단계에서 왜곡이 발생할 수 있음
- 스키마 강제성 불안정
 - JSON 모드나 구조화된 출력(Structured Outputs)을 사용하더라도 예외적인 프롬프트나 장문 컨텍스트에서는 구문 오류(Syntax Error)나 스키마 불일치가 간헐적으로 발생
- 프롬프트 드리프트(Prompt Drift)
 - 상용 파운데이션 모델(OpenAI, Anthropic 등)의 버전 업데이트나 미세 조정에 따라 기존에 잘 동작하던 프롬프트가 예기치 않게 오작동할 위험이 있음

LLM 기반 AI 서비스 개발의 주요 어려움

정량적 평가 및 CI/CD 구축의 난해함

- 단순 Pass/Fail 테스트 불가
 - "답변이 유용한가?", "톤앤매너가 적절한가?", "문맥을 충실히 반영했는가?" 같은 정성적 요소를 자동화된 파이프라인으로 측정하기 어려움
- 평가 프레임워크 구축 비용
 - 기존 NLP 메트릭의 한계로 인해, 고성능 모델을 평가자로 쓰는 LLM-as-a-Judge, Faithfulness/Answer Relevance 등을 평가하는 RAGAS 프레임워크 등을 도입해야 하며, 이 과정 자체에 추가 비용과 레이턴시가 발생

LLM 기반 AI 서비스 개발의 주요 어려움

지연 시간(Latency) 및 인프라 비용 트레이드오프

- 추론 시간의 물리적 한계
 - 일반 REST API가 수십~수백 밀리초 단위로 응답하는 반면, LLM 추론은 토큰 수와 디코딩 속도에 따라 수 초에서 수십 초가 소요
 - 이를 해결하기 위해 서버 전송 이벤트(SSE) 기반의 스트리밍 UI 설계와 비동기 큐 처리가 필수
- 운영 비용 예측의 불안정성
 - 사용자 트래픽의 단순 호출 횟수가 아니라 '입출력 토큰 총량'에 따라 과금되므로, 프롬프트 최적화 실패나 무한 루프 에이전트 발생 시 비용이 기하급수적으로 폭증

LLM 기반 AI 서비스 개발의 주요 어려움

보안 및 신뢰성(Safety) 취약점

- 프롬프트 인젝션(Prompt Injection)
 - 시스템 지침을 무력화하거나 비인가 데이터를 탈취하려는 사용자 입력(간접 인젝션 포함)을 기존 방화벽이나 정규식만으로 원천 차단하기 어려움
- 데이터 프라이버시 및 거버넌스
 - 사용자 입력에 포함된 개인정보(PII)의 외부 API 전송 통제, 기업 내부 기밀 데이터의 모델 학습 격리, 인메모리 컨텍스트 누출 방지 등의 거버넌스 설계가 복잡

LLM 기반 AI 서비스 개발의 주요 어려움

복잡해진 복구/디버깅(Observability)

- 전통적인 버그는 스택 트레이스를 추적해 코드 라인을 특정할 수 있음
- LLM 서비스의 오류는 "어떤 검색 청크가 주입되었는지", "토큰화 과정에서 손실이 있었는지", "시스템 프롬프트의 어떤 제약이 모델 추론을 왜곡했는지" 복합적인 요인으로 발생
- 전방위적인 트레이싱 도구(LangSmith 등) 없이는 근본 원인 분석이 매우 어려움

LLM 기반 AI 서비스 개발 방법론

LLM 기반 AI 서비스 개발은 고정된 로직을 작성하는 전통적 워크플로우와 달리, 프롬프트·데이터 증강·모델 튜닝 간의 상호작용 및 지속적 평가를 중심으로 진행

문제 정의 및 아키텍처 스펙트럼 선택

요구사항의 복잡도, 데이터 도메인의 폐쇄성, 비용 및 레이턴시 제약을 고려하여 최적의 기술 방식을 결정

- Prompt Engineering: 기본 모델의 인컨텍스트 러닝(In-context Learning) 역량을 활용한 제로샷/퓨샷 설계
- RAG (Retrieval-Augmented Generation): 실시간 데이터, 최신 지식, 사내 보안 문서 등을 검색 인덱스에서 조회하여 컨텍스트에 주입
- Fine-Tuning: 특정 말투(Tone & Manner), 고정된 JSON 출력 스키마, 소형 언어 모델(sLLM)의 전문 도메인 지식 내재화 필요 시 수행
- AI Agent: 단일 응답을 넘어 도구 호출(Tool Calling), 다단계 계획(Planning), 분기 및 루프(Loop)를 통한 목표 달성형 구조 설계

LLM 기반 AI 서비스 개발 방법론

데이터 엔지니어링 및 파이프라인 구축

- 정형/비정형 데이터 가공: PDF, 마크다운, DB 레코드 등의 텍스트 청킹(Chunking) 및 노이즈 제거
- 임베딩 및 인덱싱: 도메인 적합 임베딩 모델 선정, 메타데이터 필터링 구조 설계, 벡터 DB(Vector DB) 및 키워드 기반 하이브리드 검색(Hybrid Search) 적용

LLM 기반 AI 서비스 개발 방법론

관측 가능성(Observability) 및 컨텍스트 제어

- 입출력 모니터링: 시스템 프롬프트, 도구 실행 결과, 사용자 입력을 단계별로 추적하는 트레이싱(Tracing) 구축 (LangSmith 등)
- 컨텍스트 윈도우 관리: 대화 세션 누적에 따른 토큰 초과 방지, 동적 청크 필터링, 대화 요약(Summarization) 기법 적용

LLM 기반 AI 서비스 개발 방법론

LLMOps 및 정량적 평가 (Evaluation)

- 골든 데이터셋(Golden Dataset) 구축: 도메인별 표준 질의응답 세트 구성
- 다계층 평가 지표 도입: 단순 구문 일치(BLEU/ROUGE)를 넘어 답변 충실도(Faithfulness), 관련성(Answer Relevance), 환각 여부를 검증하는 RAGAS 메트릭 및 LLM-as-a-Judge 파이프라인 구성
- 회귀 테스트 자동화: 프롬프트나 인덱싱 전략 수정 시 기존 평가 세트에 미치는 영향을 지속 검증하는 CI/CD 통합

주요 사용 개발 Framework별 장단점

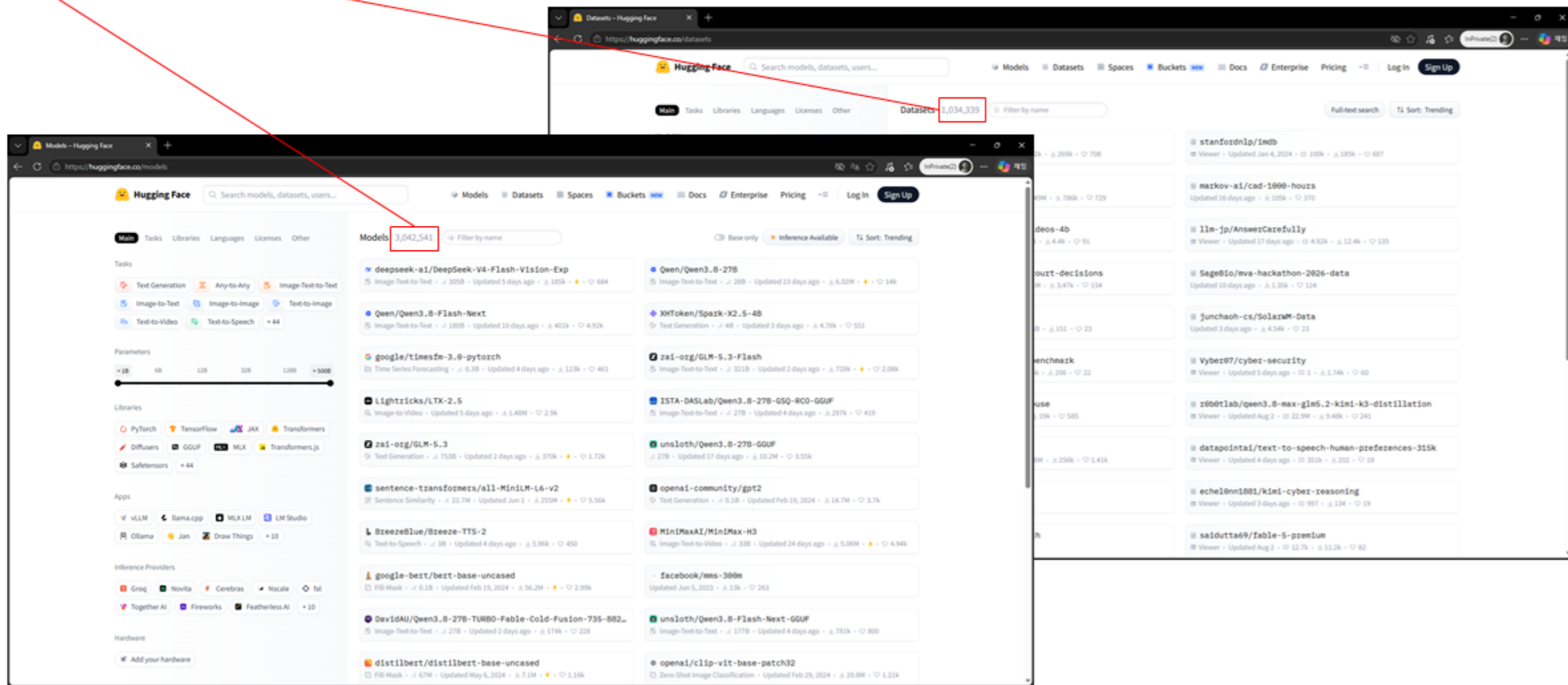
Framework	주 용도 및 핵심 개념	장점	단점
LangChain	범용 LLM 오케스트레이션 (Chains, LCEL)	• 방대한 생태계 및 모델/DB/도구 연동 컴포넌트 보유 • 빠른 프로토타이핑 및 다양한 통합 기능	• 과도한 추상화 레이어로 인한 내부 디버깅 난이도 증가 • 복잡한 다단계 에이전트 제어의 한계
LangGraph	상태 기반 순환 에이전트 (Directed Graph, State Machine)	• 노드/엣지 기반의 명시적 분기 및 루프 제어 용이 • 영속성(Persistence)과 체크포인트를 통한 휴먼인더루프(Human-in-the-loop) 구현 탁월	• 상태 정의 및 그래프 모델링에 따른 초기 학습 곡선 존재 • 단순 선형 체인 구현에는 오버엔지니어링 가능성
LlamaIndex	데이터 수집/인덱싱 및 심층 RAG (Data Ingestion, Workflows)	• 고급 청킹 전략, 계층형 인덱스 등 데이터 연동에 특화 • 구조화/비구조화 데이터를 LLM과 결합하는 RAG 성능 우수	• 비즈니스 로직 및 범용 다중에이전트 오케스트레이션 기능은 상대적으로 보조적 • 프레임워크 자체의 API 변화 빈도
CrewAI	역할 기반 멀티 에이전트 협업 (Role, Task, Crew)	• 직관적인 페르소나 설정 및 빠른 협업 워크플로우 구성 • 역할 분담 기반의 작업(조사, 요약, 검수 등) 구현 용이	• 에이전트 간 불필요한 토큰 소모 및 호출 루프 발생 위험 • 미세한 상태 조건 제어 및 엄격한 결정론적 분기 처리 제약
각 Vendor의 전용 SDK (OpenAI, Google 등)	특정 Vendor의 Model 사용 시 최적화	• 자사 모델 사용에 최적화 • 자사 타 제품군과의 연동 시 편리함, 호환성 등	• 모델 변경 시 구현 변경 어려움

프레임워크 선택 가이드라인

- RAG 파이프라인의 완성도 및 데이터 커넥터가 핵심인 경우: LlamaIndex를 중심으로 파이프라인을 설계하고 인덱싱 품질을 극대화하는 방식이 적합
- 상태 제어, 오류 복구, 사용자 승인 단계가 필요한 복잡한 에이전트인 경우: LangGraph를 도입하여 명시적 그래프 구조로 워크플로우를 통제하는 방식이 유리
- 문서 요약, 리서치, 보고서 작성 등 분업화된 역할 수행인 경우: CrewAI를 활용해 빠른 프로토타입을 제작하는 방식이 효과적
- 현업 프로덕션 아키텍처의 일반적 패턴: 단일 프레임워크에 종속되기보다 LangChain + LangGraph(상태 관리 및 오케스트레이션) 형태의 하이브리드 조합 또는 모델 네이티브 Tool Calling API와 경량 오케스트레이터를 혼합하는 방식을 널리 활용

Hugging Face

300만 개 이상의 모델과 100만 개 이상의 데이터셋을 서비스 중인 최대의 AI Hub 사이트



OpenAI

Chat Model, TTS 기능, Embedding 테스트

The screenshot shows the OpenAI Developers website's 'Developer quickstart' page. The page is designed for developers looking to integrate OpenAI's API into their applications. It features a clean, modern layout with a dark header and a light main content area. The left sidebar contains a navigation menu with categories like 'Get started', 'Core concepts', 'SDKs and CLI', and 'Resources'. The main content area is titled 'Developer quickstart' and includes a sub-header 'Take your first steps with the OpenAI API.' Below this, there's a paragraph explaining the API's capabilities. A prominent call-to-action box encourages users to 'Build with the OpenAI API in ChatGPT and Codex' and provides a link to 'Install the plugin'. The 'Create and export an API key' section is highlighted, showing a 'Create an API Key' button and instructions on how to securely store and use the key. A code block displays the command to export the API key as an environment variable for macOS or Linux. The page also includes a 'Copy Page' button and an 'Ask AI' button in the bottom right corner.

OpenAI Developers

Home API Codex ChatGPT Resources

Start searching

API Dashboard

Overview Models Agents Tools Voice & Audio Production API reference

Home

Get started

Quickstart

Using GPT-6 Astra

Key concepts

Core concepts

Responses API

Conversation state

Background mode

Streaming

WebSocket mode

Mid-turn steering

Multi-agent

Webhooks

File inputs

Compaction

Counting tokens

SDKs and CLI

OpenAI SDK

OpenAI CLI

Resources

Developer quickstart

Take your first steps with the OpenAI API.

The OpenAI API provides a consistent interface to state-of-the-art AI models for text generation, natural language processing, computer vision, and more. Get started by creating an API Key and running your first API call. Discover how to generate text, analyze images, build agents, and more.

Build with the OpenAI API in ChatGPT and Codex

The OpenAI Developers plugin connects ChatGPT and Codex to the OpenAI Platform, follows OpenAI API setup guidance, and creates project API keys when your application needs one.

Install the plugin

Create and export an API key

Create an API Key

Before you begin, create an API key in the dashboard, which you'll use to securely [access the API](#). Store the key in a safe location, like a [.zshrc file](#) or another text file on your computer. Once you've generated an API key, export it as an [environment variable](#) in your terminal.

macOS / Linux Windows

Export an environment variable on macOS or Linux systems

```
export OPENAI_API_KEY="your_api_key_here"
```

Each OpenAI SDK automatically reads your API key from the system environment.

Install the OpenAI SDK and Run an API Call

Install the OpenAI SDK and Run an API Call

Ask AI

Google GenAI

생성형 AI, 대화 생성 테스트

The screenshot shows the Gemini API documentation page in Korean. The page is titled "Gemini API" and includes a sidebar with navigation links. The main content area features a blue banner with a message about the Interactions API, a "Switch to English" button, and a code snippet in Python. Below the code, there is a "Model 살펴보기" (View Models) section with buttons for Gemini 3.1 Pro, Gemini 3.7 Flash, and Gemini 3.6 Flash.

문서 API 참조 문서

시작하기

개요

시작하기

API 키

가격 책정

로딩 에이전트 설정

모델

모든 모델

최신 Gemini 모델

Nano Banana

Veo

Gemini Omni Flash

Lyria 3.5

Lyria RealTime

Imagen

TTS(text-to-speech)

스크립트 작성

게시물

실시간 번역

임베딩

로봇공학 >

핵심 기능

텍스트

이미지 >

동영상 >

이제 **Interactions API**가 정식 버전으로 출시되었습니다. 이 API를 사용하여 모든 최신 기능과 모델에 액세스하는 것이 좋습니다.

translated by Google Google은 AI 기술을 사용하여 콘텐츠를 사용자의 기본 언어로 번역합니다. AI 번역에는 오류가 있을 수 있습니다. [Switch to English](#)

홈 > Gemini API > 문서

Gemini API

Gemini API는 Gemini, Veo, Nano Banana 등을 사용하여 프롬프트에서 프로덕션까지 도달하는 가장 빠른 경로를 제공합니다. 이 API를 사용하면 이러한 생성형 모델을 애플리케이션에 통합하여 텍스트와 이미지를 생성하고, 멀티모달 입력을 분석하고, 대화형 에이전트를 빌드할 수 있습니다.

[Python](#) [JavaScript](#) [REST](#)

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.7-flash",
    input="Explain how AI works in a few words"
)

print(interaction.output_text)
```

[레고 시작하기](#) | 시작하기 가이드에 따라 API 키를 가져오고 몇 분 만에 첫 번째 API 호출을 실행하세요.

모델 살펴보기

모두 보기

[Gemini 3.1 Pro](#) [신규](#) [Gemini 3.7 Flash](#) [신규](#) [Gemini 3.6 Flash](#)

LangChain 컴포넌트

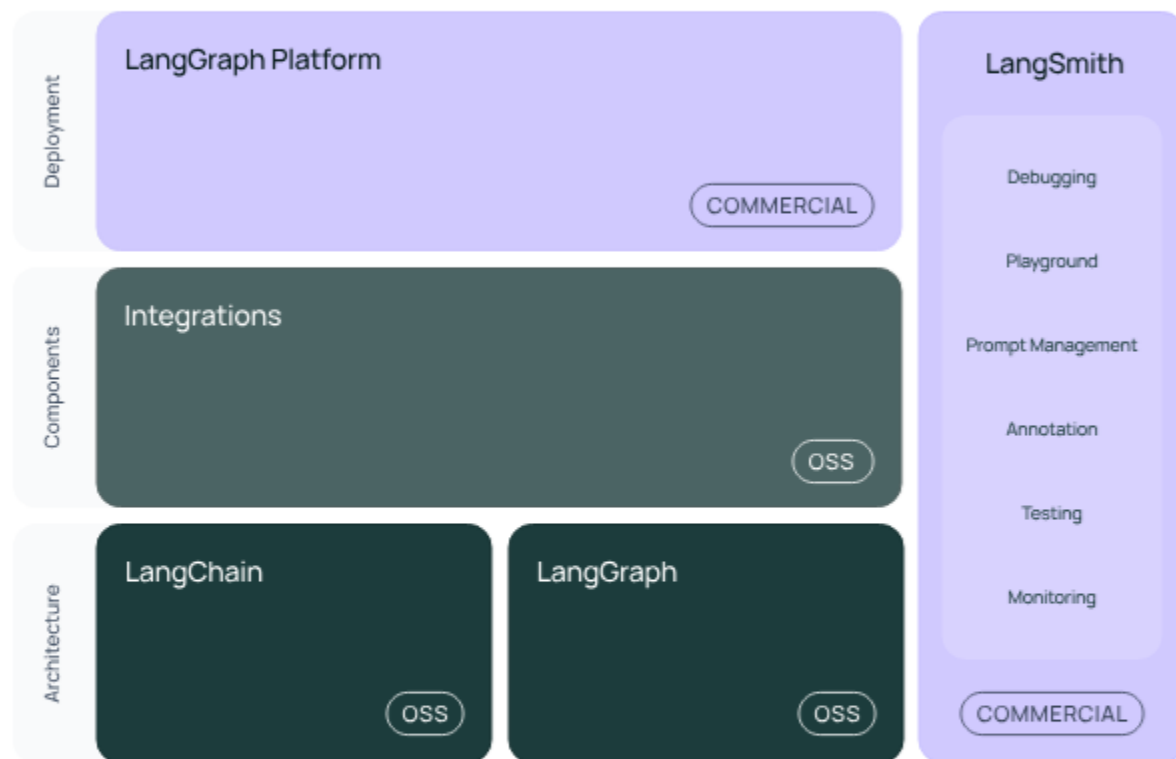
- LangChain Introduction
- LangChain 기본 컴포넌트

LangChain Introduction

- langchain
 - 애플리케이션의 인지 구조를 구성하기 위한 체인과 추출 전략을 포함
 - 제 3자(third-party) 통합체는 아님
 - 모든 체인, 에이전트, 추출 전략은 특정 통합체에 속하는 것이 아닌 모든 통합체의 일반적인 것에 해당
- langchain-core
 - chat models, vector stores, tools과 같은 핵심 컴포넌트를 위한 인터페이스를 정의
 - 제 3자(third-party) 통합체가 정의되지는 않음
 - 의존성은 경량으로 유지됨
- 적절한 버전관리와 경량화를 달성하기 위해 대중적인 통합체는 자체적인 패키지 (e.g. langchain-openai, langchain-anthropic, etc) 를 보유하고 있음

LangChain Introduction

- Integrations
 - LangChain = 다양한 AI 서비스와 도구를 연결하는 통합 생태계
 - Provider = AI 모델을 제공하는 회사 또는 플랫폼
 - `langchain-<provider>` 패키지를 통해 각 provider를 LangChain 방식으로 연결
 - 표준 인터페이스 덕분에 OpenAI → Anthropic → Google 처럼 provider를 쉽게 교체 가능
 - 핵심 장점: provider가 달라도 코드 구조를 일관되게 유지할 수 있음



LangChain 기본 컴포넌트 소개

LangChain은 다음의 컴포넌트들을 조합해 AI 서비스를 만드는 프레임워크

- Chat Model: LangChain에서 다양한 LLM을 다루는 표준 인터페이스
- Messages: 대화를 구성하는 메시지 객체 체계
- Prompt Template: LLM을 위한 Prompt 생성
- Short-term memory: 단일 세션 내 대화 기억
- Streaming: 실시간 응답 처리 방식
- Structured Output: 정형화된 형식으로 답변 받기
- LCEL: 컴포넌트를 연결하는 표현식
- Agent: 스스로 판단하여 행동하는 LLM 기반 실행 단위
- Tools: LLM이 외부 기능을 호출하도록 돕는 도구

Model이란 무엇인가?

핵심 개념

- Model: LangChain에서 LLM을 호출하는 표준화된 인터페이스
- OpenAI, Anthropic, Google 등 다양한 제공사의 모델을 동일한 방식으로 사용
- `init_chat_model` 함수로 모델 문자열만 바꿔 손쉽게 교체 가능
- 온도(temperature), 최대 토큰 등 생성 파라미터 설정 가능
- Chat 모델과 완성형(Completion) 모델로 구분되며, 현재는 Chat 모델이 표준

Model 인터페이스의 장점

상세 설명

- 특정 벤더에 종속되지 않는 벤더 중립적 설계
- 모델 교체 시 비즈니스 로직 코드를 거의 수정하지 않아도 됨
- Runnable 인터페이스를 공유해 체인(Chain) 구성에 바로 활용 가능
- 동기(sync)와 비동기(async) 호출을 모두 지원
- 재시도(retry), 캐싱 등 부가 기능도 표준 인터페이스로 제공

Model 파라미터 이해

상세 설명

파라미터	설명
temperature	답변의 창의성/무작위성 조절 (0~2)
max_tokens	생성할 최대 토큰 수
top_p	확률 분포 상위 비율만 고려
model	사용할 모델 이름

- temperature가 낮을수록 일관되고 보수적인 답변
- temperature가 높을수록 다양하고 창의적인 답변

Model 초기화 예제

예제

```
1 from langchain.chat_models import init_chat_model
2
3 model = init_chat_model(
4     "gpt-4",
5     model_provider="openai",
6     temperature=0.7
7 )
8
9 response = model.invoke("LangChain의 장점을 설명해줘")
10 print(response.content)
11
```

1. 다양한 언어 지원: LangChain은 24개 이상의 언어를 지원합니다. 이를 통해 다양한 언어 간의 통역이나 번역을 용이하게 할 수 있습니다.
2. AI 기술 활용: LangChain은 최신 인공지능 기술을 활용하여 더욱 정확하고 자연스러운 번역 결과를 제공합니다.
3. 빠른 번역 속도: LangChain은 빠른 번역 속도를 자랑합니다. 이를 통해 사용자는 즉시 번역 결과를 확인할 수 있습니다.
4. 사용자 친화적인 인터페이스: LangChain은 사용자 친화적인 인터페이스를 제공하여, 누구나 쉽게 사용할 수 있습니다.
5. 다양한 사용 케이스: LangChain은 개인적인 용도부터 비즈니스 용도까지 다양한 사용 케이스를 지원합니다. 이를 통해 사용자는 자신의 상황에 맞게 활용할 수 있습니다.
6. 안전한 데이터 보호: LangChain은 사용자의 개인 정보와 데이터를 보호하기 위해 다양한 보안 기술을 적용하고 있습니다. 이를 통해 사용자는 안심하고 사용할 수 있습니다.

다양한 모델 교체 실습

예제

```
1 # OpenAI 모델
2 model_openai = init_chat_model("gpt-4", model_provider="openai")
3
4 # Google 모델
5 model_google = init_chat_model("gemini-2.5-flash-lite", model_provider="google_genai")
6
7 for m in [model_openai, model_google]:
8     print(m.invoke("한 문장으로 자기소개 해줘").content + "\n\n")
9
```

안녕하세요, 저는 OpenAI의 인공지능 비서로, 다양한 정보 제공, 예약, 일정 관리 등을 통해 사용자의 일상을 도와주는

저는 끊임없이 배우고 성장하며, 긍정적인 에너지를 나누는 사람입니다.

Model 활용 팁

상세 설명

- 프로토타입 단계에서는 저렴하고 빠른 모델로 먼저 검증
- 정확도가 중요한 단계에서는 고성능 모델로 전환
- 반복 호출이 많은 서비스는 캐싱(Caching)으로 비용 절감
- 모델별 강점(추론, 코드, 멀티모달 등)을 파악해 목적에 맞게 선택

Messages란 무엇인가?

핵심 개념

- Messages: LLM과의 대화를 구성하는 표준화된 메시지 객체
- 역할(role)에 따라 여러 종류로 구분됨
- SystemMessage: 모델의 행동 지침 설정
- HumanMessage: 사용자의 발화
- AIMessage: 모델의 응답
- ToolMessage: 도구 실행 결과 전달용 메시지

Messages의 역할별 구조

상세 설명

메시지 타입	역할
SystemMessage	모델의 페르소나, 규칙 지정
HumanMessage	사용자 입력
AIMessage	모델의 응답 (Tool 호출 포함 가능)
ToolMessage	도구 실행 결과 반환

- 메시지들은 리스트 형태로 순서대로 모델에 전달됨
- 이 순서가 곧 대화의 문맥(Context)을 형성

Messages 활용 예제

예제

```
1 from langchain_core.messages import SystemMessage, HumanMessage
2
3 messages = [
4     SystemMessage(content="당신은 친절한 프로그래밍 튜터입니다."),
5     HumanMessage(content="파이썬 리스트와 튜플의 차이를 알려줘")
6 ]
7
8 response = model.invoke(messages)
9 print(response.content)
10
```

파이썬에서 리스트와 튜플은 비슷해 보일 수 있지만, 중요한 차이점이 몇 가지 있습니다.

1. 가변성 (Mutability): 리스트는 가변적이다. 즉, 리스트의 요소는 생성 후에 변경, 추가, 삭제가 가능합니다. 반면에
2. 문법: 리스트는 대괄호([])를 사용하여 요소를 감싸고, 각 요소는 콤마(,)로 구분됩니다. 반면에 튜플은 괄호(())를
3. 사용 경우: 리스트는 요소의 순서가 중요하고, 요소를 동적으로 업데이트해야 하는 경우에 사용됩니다. 튜플은 요소
4. 메모리 및 성능: 튜플은 리스트와 비교하여 더 적은 메모리를 사용하고 처리 속도가 더 빠릅니다. 따라서 요소의 크

AIMessage와 Tool 호출

상세 설명

- AIMessage는 단순 텍스트뿐 아니라 tool_calls 정보를 포함할 수 있음
- 모델이 도구 호출이 필요하다고 판단하면 tool_calls 필드에 호출 정보 기록
- 이후 해당 도구를 실행하고 결과를 ToolMessage로 다시 전달
- 이 과정이 반복되며 Agent의 사고-행동 루프가 형성됨

```
print(response.tool_calls)
# [{'name': 'get_weather', 'args': {'city': '서울'}, 'id': 'call_123'}]
```

Messages 관리 시 유의점

상세 설명

- 메시지 리스트가 길어지면 Context Window 제한에 유의
- 시스템 메시지는 대화 전체에 영향을 주므로 신중하게 설계
- 사용자별/세션별로 메시지 기록을 분리 관리해야 함
- 메시지 직렬화(JSON 변환) 시 타입 정보 손실에 주의

대화 흐름 구성 실습

예제

```
1 conversation = []
2 conversation.append(SystemMessage(content="당신은 요리 전문가입니다."))
3 conversation.append(HumanMessage(content="김치찌개 레시피 알려줘"))
4
5 response = model.invoke(conversation)
6 conversation.append(response)
7
8 conversation.append(HumanMessage(content="더 맵게 하려면?"))
9 response2 = model.invoke(conversation)
10 print(response2.content)
11
```

더 맵게 하려면 아래와 같은 방법들을 시도해 볼 수 있습니다:

1. 고춧가루를 더 넣습니다: 레시피에서 제시한 양보다 더 많은 고춧가루를 넣으면 매운맛이 강해집니다.
2. 청양고추를 더 넣습니다: 청양고추는 매운맛이 강한 편이므로, 김치찌개에 청양고추를 더 추가하면 매운맛이 더욱 강해집니다.
3. 고추장을 추가합니다: 고추장은 매운맛을 더하면서도 깊은 맛을 줍니다. 1-2큰술 정도를 추가해 보세요.
4. 김치 양을 늘립니다: 김치 자체가 매콤한 맛을 가지고 있으므로 김치의 양을 늘리면 매운맛이 더해집니다.
5. 매운 고추기름을 사용합니다: 고추기름을 사용하면 매운맛을 더할 수 있습니다. 들기름 대신 매운 고추기름을 사용하면 매운 음식을 좋아하신다면 이런 방법들을 활용해 보세요. 하지만 너무 많이 넣으면 맛이 치중되어 김치찌개의 본연의 맛을 잃을 수 있습니다.

Prompt Template이란 무엇인가?

핵심 개념

- Prompt Template: 변수만 바뀌가며 재사용할 수 있는 프롬프트 틀(템플릿)
- 매번 프롬프트 문자열을 직접 조합하지 않고, 중괄호 변수(`{ }`)로 자리표시
- 사용자 입력, 검색 결과 등 동적인 값을 템플릿에 끼워 넣어 완성된 프롬프트 생성
- LangChain의 `ChatPromptTemplate` , `PromptTemplate` 클래스로 구현
- 프롬프트를 코드와 분리해 관리할 수 있어 유지보수성이 크게 향상됨
- 여러 개발자가 동일한 프롬프트 품질을 유지하도록 돕는 표준화 도구

Prompt Template의 구조

상세 설명

- PromptTemplate: 단일 문자열 기반의 단순 템플릿 (완성형 모델용)
- ChatPromptTemplate: 여러 메시지(System/Human/AI)로 구성된 템플릿 (Chat 모델 표준)
- 템플릿 내 변수는 `{variable_name}` 형태로 작성
- `.invoke()` 또는 `.format()` 에 딕셔너리를 전달해 변수 값을 채움
- System 메시지를 템플릿 안에 고정해 모델의 역할/톤을 일관되게 유지 가능

구성 요소	역할
System 부분	모델의 페르소나, 규칙 지정
Human 부분	사용자 입력이 들어가는 자리
변수({})	실행 시점에 채워지는 동적 값

Prompt Template 기본 실습

예제

```
1 from langchain_core.prompts import ChatPromptTemplate
2
3 prompt = ChatPromptTemplate.from_messages([
4     ("system", "당신은 {role} 전문가입니다. 친절하고 간결하게 답변하세요."),
5     ("human", "{question}")
6 ])
7
8 formatted = prompt.invoke({"role": "영양", "question": "다이어트에 좋은 저녁 메뉴 추천해주"})
9 print(formatted.to_messages())
10
```

```
[SystemMessage(content='당신은 영양 전문가입니다. 친절하고 간결하게 답변하세요.', additional_kwargs={}, response_metadata={})]
```

```
[SystemMessage(content='당신은 영양 전문가입니다. 친절하고 간결하게 답변하세요.', additional_kwargs={}, response_metadata={}), HumanMessage(content='다이어트에 좋은 저녁 메뉴 추천해줘', additional_kwargs={}, response_metadata={})]
```

Prompt Template 작성 팁

상세 설명

- 변수명은 의미가 명확하게 드러나도록 작성 (`{q}` 보다 `{question}`)
- System 메시지에는 역할, 톤, 제약 조건을 구체적으로 명시
- 자주 쓰는 템플릿은 별도 파일/모듈로 분리해 재사용성 확보
- 템플릿 변경 이력을 관리해 프롬프트 품질을 지속적으로 개선

Short-term memory란 무엇인가?

핵심 개념

- Short-term memory: 하나의 대화 세션 내에서 이전 메시지를 기억하는 기능
- 사용자가 이전에 한 말을 참고해 일관된 대화 유지
- 세션이 종료되거나 초기화되면 사라지는 일시적 기억
- 장기 기억(Long-term memory)과 구분되는 개념

Short-term memory의 필요성

상세 설명

- 메모리가 없으면 매 요청마다 문맥 없는 독립적인 대화가 됨
- "그거 다시 설명해줘"와 같은 지시어(대명사) 이해 불가
- 멀티턴 대화형 서비스(챗봇 등)에는 필수적인 기능
- 다만 무제한 저장 시 Context Window 초과 문제 발생 가능

Short-term memory 구현 방식

상세 설명

- 메시지 리스트 누적: 가장 단순한 방식, 모든 메시지를 계속 추가
- Checkpointer 기반 상태 저장: LangGraph에서 세션별 상태를 자동 저장
- Thread ID: 각 대화 세션을 구분하는 고유 식별자
- 동일한 Thread ID로 호출하면 이전 대화 상태를 불러와 이어감

```
config = {"configurable": {"thread_id": "user-123"}}
```

Short-term memory 실습

예제

```
1 from langgraph.checkpoint.memory import InMemorySaver
2 from langchain.agents import create_agent
3
4 checkpointer = InMemorySaver()
5 agent = create_agent(model=model, tools=[], checkpointer=checkpointer)
6
7 config = {"configurable": {"thread_id": "session-1"}}
8
9 agent.invoke({"messages": [{"role": "user", "content": "내 이름은 민준이야"}]}, config)
10 result = agent.invoke({"messages": [{"role": "user", "content": "내 이름이 뭐라고 했지?"}]}
11 print(result["messages"][-1].content)
12
```

당신의 이름은 민준이라고 말씀하셨습니다.

메모리 관리 시 주의점

상세 설명

- 세션별 메모리를 적절히 분리하지 않으면 다른 사용자 정보가 섞일 위험
- 저장된 메모리가 커지면 비용과 지연시간 증가
- 민감 정보(개인정보 등)는 메모리 저장 시 보안 고려 필요
- 필요 시 오래된 메시지를 요약하거나 삭제하는 정책 수립

실무 적용 사례

예제

- 고객 상담 챗봇: 세션 내내 고객의 문의 이력 기억
- 개인 비서 AI: 사용자의 선호와 이전 요청 기억
- 교육용 챗봇: 학생의 이전 질문 수준을 파악해 난이도 조절

Streaming이란 무엇인가?

핵심 개념

- Streaming: 모델의 응답을 한 번에 받지 않고 토큰 단위로 순차 전달받는 방식
- 사용자는 답변이 생성되는 과정을 실시간으로 확인 가능 (ChatGPT의 타이핑 효과)
- 긴 답변일수록 체감 응답 속도가 크게 개선됨
- LangChain에서는 `stream()` 메서드로 간단히 구현

Streaming의 필요성

상세 설명

- 전체 응답 생성까지 기다리면 긴 대기 시간 발생
- 스트리밍을 사용하면 첫 토큰이 도착하는 즉시 화면에 표시 가능
- 사용자 경험(UX) 측면에서 매우 중요한 기능
- 챗봇, 실시간 협업 도구 등에서 표준적으로 사용됨

Streaming 실습

예제

```
1 for chunk in model.stream("인공지능의 역사를 3문단으로 설명해줘"):
2     | print(chunk.content, end="", flush=True)
3
```

인공지능의 역사는 20세기 중반에 시작되었습니다. 1956년, 존 매카시, 앨런 뉴얼, 허버트 사이먼, 그리고 마빈 민스키
그러나 1970년대가 되면서 인공지능의 한계와 문제가 부각되었고, 이를 '시의 겨울'이라 부르기도 합니다. 연구자들은
1990년대부터는 머신 러닝, 딥 러닝 등의 개발과 함께 인공지능의 '봄'이 도래하였습니다. 빅데이터와 컴퓨터의 계산 능

비동기 Streaming

상세 설명

- `astream()` 을 사용하면 비동기 방식으로 스트리밍 처리 가능
- 웹 서버(FastAPI 등)에서 여러 사용자 요청을 동시에 처리할 때 유용
- 비동기 스트리밍은 서버 자원을 효율적으로 활용

```
1 async for chunk in model.astream("파이썬의 장점을 알려줘"):
2     print(chunk.content, end="")
```

1. 간결하고 읽기 쉬운 코드: 파이썬은 간결하고 명확한 문법을 가지고 있어 코드를 이해하고 작성하는 것이 더 쉽습니다.
2. 다양한 분야에서 활용 가능: 파이썬은 웹 개발, 데이터 분석, 인공지능, 기계 학습, 딥 러닝 등 다양한 분야에서 활용됩니다.
3. 강력한 라이브러리와 프레임워크: 넘파이, 판다스, 맷플롯립 등 데이터 분석 라이브러리와 장고, 플라스크 등의 웹 프레임워크가 있습니다.
4. 동적 타이핑 지원: 컴파일 시점이 아닌 런타임 시점에 데이터 타입을 결정하므로 개발 과정이 더 유연하고 빠릅니다.
5. 커뮤니티 지원: 전 세계 많은 개발자들이 사용하고 있어, 다양한 정보를 얻을 수 있고, 문제가 생겼을 때 도움을 받을 수 있습니다.
6. 좋은 생산성: 파이썬의 간결하고 직관적인 문법은 개발 시간을 단축시키고 생산성을 향상시킵니다.
7. 학습이 비교적 쉽다: 문법이 직관적이고 간결하여 프로그래밍을 처음 시작하는 사람들이 배우기 쉽습니다.

Streaming 적용 시 고려사항

상세 설명

- 프론트엔드(UI)에서 점진적 렌더링 처리 필요
- 네트워크 연결(WebSocket, SSE 등) 설계 필요
- 스트리밍 도중 오류 발생 시 부분 응답 처리 전략 필요
- Structured Output과 함께 사용할 때는 파싱 방식에 유의

Structured Output이란 무엇인가?

핵심 개념

- Structured Output: LLM의 응답을 자유 텍스트가 아닌 정해진 데이터 구조로 받는 방식
- JSON, Pydantic 모델 등 원하는 스키마에 맞춰 응답 생성
- 애플리케이션에서 응답을 파싱 없이 바로 활용 가능
- `with_structured_output()` 메서드로 간단히 적용

Structured Output이 필요한 이유

상세 설명

- 자유 텍스트 응답은 일관된 파싱이 어려움
- 예: "이름: 홍길동, 나이: 20살" 형태는 오타나 순서 변화에 취약
- 정형화된 출력은 프로그램에서 바로 필드 단위로 접근 가능
- API 연동, 데이터베이스 저장 등에 필수적인 기능

Structured Output 실습

예제

```
1 from pydantic import BaseModel
2
3 class PersonInfo(BaseModel):
4     name: str
5     age: int
6     job: str
7
8 structured_model = model.with_structured_output(PersonInfo)
9 result = structured_model.invoke(
10     "홍길동은 28살이고 개발자로 일하고 있습니다. 정보를 추출해줘"
11 )
12
13 print(result)
14 print(result.name, result.age, result.job)
```

```
name='홍길동' age=28 job='개발자'
홍길동 28 개발자
```

Structured Output의 내부 동작

상세 설명

- 대부분 Function Calling(도구 호출) 방식을 내부적으로 활용
- 지정한 스키마를 하나의 "도구"처럼 모델에 전달
- 모델은 텍스트 응답 대신 스키마에 맞는 구조화된 인자를 반환
- 일부 모델은 JSON 모드를 직접 지원하기도 함

Structured Output 활용 사례

예제

- 이력서 파싱: 자유 형식 이력서에서 이름/경력/기술 스택 추출
- 주문 정보 추출: 고객 문의에서 상품명/수량/배송지 추출
- 감정 분석: 텍스트에서 감정 라벨과 신뢰도 점수 추출
- 폼 자동 작성: 대화 내용을 기반으로 정형 폼 데이터 생성

Output Parser와 Structured Output

- Pydantic: 데이터 검증과 스키마 정의를 위한 라이브러리
- JSON: 구조화된 데이터 교환 형식
- Schema: 응답의 형태를 정의하는 설계도
- Validation: 생성된 데이터가 규칙에 맞는지 검증하는 과정

Output Parser 개요

핵심 개념

- Output Parser: LLM의 텍스트 응답을 원하는 데이터 형태로 변환하는 컴포넌트
- 가장 단순한 형태는 `StrOutputParser` (텍스트만 추출)
- 고급 형태는 JSON, Pydantic 객체 등으로 파싱
- LCEL 체인의 마지막 단계에 주로 위치 (`prompt | model | parser`)

Pydantic이란 무엇인가?

핵심 개념

- Pydantic: Python의 데이터 검증 및 설정 관리 라이브러리
- 클래스 기반으로 데이터 구조(스키마)를 정의
- 타입 힌트를 활용해 자동으로 데이터 유효성 검사 수행
- LangChain의 Structured Output 기능의 핵심 기반 라이브러리
- FastAPI 등 다른 유명 프레임워크에서도 널리 사용됨

Pydantic 모델 정의하기

상세 설명

- BaseModel 을 상속받아 클래스 형태로 데이터 구조 정의
- 각 필드는 타입과 함께 설명(Field description)을 추가할 수 있음
- 이 설명은 LLM이 어떤 값을 채워야 하는지 이해하는 데 활용됨

```
1 from pydantic import BaseModel, Field
2
3 class MovieReview(BaseModel):
4     title: str = Field(description="영화 제목")
5     rating: int = Field(description="1~5점 사이의 평점")
6     summary: str = Field(description="한 줄 감상평")
```

Pydantic과 LLM 결합 실습

예제

```
1 structured_model = model.with_structured_output(MovieReview)
2
3 result = structured_model.invoke(
4     "인터스텔라는 정말 감동적이었어요. 5점 만점에 5점 주고 싶어요"
5 )
6
7 print(result)
8 print(result.title, result.rating, result.summary)
```

```
... title='인터스텔라' rating=5 summary='정말 감동적이었어요.'
인터스텔라 5 정말 감동적이었어요.
```

Pydantic 필드 검증 기능

상세 설명

- 필드에 제약 조건(constraint)을 추가해 값의 범위 제한 가능
- 예: 평점은 반드시 1~5 사이의 값이어야 함

```
1 from pydantic import Field
2
3 class MovieReview(BaseModel):
4     rating: int = Field(ge=1, le=5, description="1~5점 사이의 평점")
5
```

- 조건을 벗어나는 값이 들어오면 자동으로 오류 발생

중첩 Pydantic 모델

상세 설명

- Pydantic 모델 안에 다른 Pydantic 모델을 필드로 포함 가능
- 복잡한 구조의 데이터도 계층적으로 표현 가능

```
1 class Actor(BaseModel):  
2     name: str  
3     role: str  
4  
5 class MovieInfo(BaseModel):  
6     title: str  
7     actors: list[Actor]  
8
```

JSON이란 무엇인가?

핵심 개념

- JSON(JavaScript Object Notation): 키-값 쌍으로 구성된 경량 데이터 교환 형식
- 대부분의 프로그래밍 언어와 API에서 표준으로 사용
- 사람이 읽기 쉬우면서도 프로그램이 파싱하기 쉬운 구조
- LLM의 Structured Output은 내부적으로 JSON 형태를 거치는 경우가 많음

JSON 기본 구조

상세 설명

```
1 {  
2   "name": "홍길동",  
3   "age": 28,  
4   "skills": ["Python", "LangChain"],  
5   "is_student": False  
6 }
```

```
{'name': '홍길동',  
 'age': 28,  
 'skills': ['Python', 'LangChain'],  
 'is_student': False}
```

- 객체(Object): 중괄호 `{}` 로 표현되는 키-값 쌍
- 배열(Array): 대괄호 `[]` 로 표현되는 목록
- 값의 타입: 문자열, 숫자, 불리언, 배열, 객체, null

LLM으로부터 JSON 응답 받기

예제

```
1 from langchain_core.output_parsers import JsonOutputParser
2
3 parser = JsonOutputParser()
4 prompt = ChatPromptTemplate.from_messages([
5     ("human", "다음의 정보로부터 이름, 나이, 직업을 json 형식으로 추출해줘. {text}")
6 ])
7
8 chain = prompt | model | parser
9
10 result = chain.invoke({"text": "홍길동, 28세, 개발자입니다"})
11 print(type(result)) # <class 'dict'>
12 print(result)
13 print(result["이름"])
14
```

```
<class 'dict'>
{'이름': '홍길동', '나이': 28, '직업': '개발자'}
홍길동
```

JSON 파싱 시 발생하는 문제

상세 설명

- LLM이 완벽한 JSON 형식이 아닌 텍스트를 생성할 위험 존재 (따옴표 누락 등)
- `JsonOutputParser` 는 일부 형식 오류를 자동으로 보정하기도 함
- 프롬프트에 "반드시 JSON 형식으로만 답하라"는 지시를 명확히 포함하는 것이 중요
- 안전성이 중요한 서비스는 Pydantic 기반 구조화 출력을 우선 권장

Schema란 무엇인가?

핵심 개념

- Schema: 데이터가 어떤 필드와 타입으로 구성되는지 정의하는 설계도
- Pydantic 모델 자체가 하나의 Schema 역할을 수행
- LLM에게 Schema를 전달하면, 해당 구조에 맞춰 응답을 생성하도록 유도
- JSON Schema는 다양한 언어/도구 간 공통 표준 형식으로도 널리 사용됨

JSON Schema 형태

상세 설명

```
1 {  
2   "title": "MovieReview",  
3   "type": "object",  
4   "properties": {  
5     "title": {"type": "string", "description": "영화 제목"},  
6     "rating": {"type": "integer", "description": "1~5점 평점"}  
7   },  
8   "required": ["title", "rating"]  
9 }
```

```
{'title': 'MovieReview',  
 'type': 'object',  
 'properties': {'title': {'type': 'string', 'description': '영화 제목'},  
 'rating': {'type': 'integer', 'description': '1~5점 평점'}},  
 'required': ['title', 'rating']}
```

- Pydantic 모델은 `.model_json_schema()` 로 이러한 JSON Schema로 변환 가능

Schema 활용 실습

예제

```
1 schema = MovieReview.model_json_schema()
2 print(schema)
3
4 # LangChain 내부적으로 이 schema가 Function Calling 형태로 모델에 전달됨
5 structured_model = model.with_structured_output(MovieReview)
```

```
{'properties': {'rating': {'description': '1~5점 사이의 평점', 'maximum': 5, 'minimum': 1, 'title': 'Rating',
```

```
{'properties':
  {'rating':
    {'description': '1~5점 사이의 평점', 'maximum': 5,
     'minimum': 1,
     'title': 'Rating',
     'type': 'integer'
    }
  }, 'required': ['rating'],
  'title': 'MovieReview',
  'type': 'object'
}
```

Schema 설계 Best Practice

상세 설명

- 필드명은 의미가 명확하게 드러나도록 작성
- 각 필드에 description을 반드시 추가해 LLM의 이해를 도움
- 너무 많은 필드보다는 꼭 필요한 필드만 포함
- 선택적(Optional) 필드와 필수(Required) 필드를 명확히 구분

LCEL이란 무엇인가?

핵심 개념

- LCEL(LangChain Expression Language): LangChain 컴포넌트를 연결하는 선언적 문법
- Prompt, Model, Parser 등을 파이프(`|`) 연산자로 순서대로 연결
- 모든 구성요소가 Runnable 인터페이스를 공유해 자유롭게 조합 가능
- `invoke()` , `stream()` , `batch()` 등 동일한 실행 방식을 모든 체인에 그대로 적용 가능
- 복잡한 파이프라인도 간결한 한 줄 코드로 표현할 수 있음

LCEL 파이프 연산자 실습

예제

```
1 from langchain_core.output_parsers import StrOutputParser
2
3 chain = prompt | model | StrOutputParser()
4
5 result = chain.invoke({"role": "여행", "question": "3박 4일 제주 일정 짜줘"})
6 print(result)
7
```

1일차:

- 오전: 제주 공항 도착 및 픽업, 제주 도시 관광 (동문시장, 이중섭 미술관)
- 오후: 제주도 유명한 해안 도로인 애월 해안도로 드라이브, 한림 공원
- 저녁: 애월해안가에서 식사

2일차:

- 오전: 성산일출봉 일출 감상 (날씨 좋을 시), 섭지코지 방문
- 오후: 세화 해변, 만장굴 탐방
- 저녁: 성산일출봉 인근에서 식사

3일차:

- 오전: 천제연 폭포, 주상절리대 방문
- 오후: 중문 관광단지, 산방산
- 저녁: 중문 관광단지 인근에서 식사

4일차:

- 오전: 제주도 특산물 구매 (올레시장, 중앙로 지하상가)
- 오후: 제주 공항으로 이동 및 도착, 출발 준비

이 일정은 날씨, 교통 상황 등에 따라 변경될 수 있으니 참고해 주세요.

LCEL의 장점

상세 설명

- 가독성: `prompt | model | parser` 처럼 실행 흐름이 코드에 그대로 드러남
- 재사용성: 각 단계를 독립적인 컴포넌트로 분리해 다른 체인에서도 재사용 가능
- 일관된 실행 방식: 어떤 체인이든 동일한 방식(`invoke/stream/batch`)으로 호출
- 병렬/순차 조합: `RunnableParallel`, `RunnableSequence`와 자유롭게 결합 가능
- 부가 기능 내장: 재시도(`with_retry`), Fallback(`with_fallbacks`) 등을 손쉽게 적용

Agent란 무엇인가?

핵심 개념

- Agent: 주어진 목표를 달성하기 위해 스스로 판단하고 행동하는 LLM 실행 단위
- 단순 질의응답과 달리 여러 단계에 걸쳐 추론과 행동을 반복
- 필요 시 Tool(도구)을 호출해 외부 정보를 얻거나 작업 수행
- 행동 결과를 관찰(Observation)한 뒤 다음 행동을 결정
- LangChain에서는 `create_agent` 등의 API로 손쉽게 구성 가능
- 복잡한 업무 프로세스를 자동화하는 데 활용됨

Agent의 동작 원리

상세 설명

- 사고(Reasoning) → 행동(Action) → 관찰(Observation)의 반복 루프
- LLM이 "다음에 어떤 도구를 호출할지" 스스로 결정
- 도구 실행 결과를 다시 LLM에 전달해 다음 판단에 활용
- 목표 달성 또는 최대 반복 횟수에 도달하면 종료
- 이러한 반복 구조를 ReAct(Reasoning + Acting) 패턴이라고도 부름

LangChain Agent 생성 예제

예제

```
1 from langchain.agents import create_agent
2 from langchain_openai import ChatOpenAI
3
4 model = ChatOpenAI(model="gpt-4")
5
6 def get_weather(city: str) -> str:
7     """도시의 날씨 정보를 반환합니다."""
8     return f"{city}는 현재 맑음, 22도입니다."
9
10 agent = create_agent(model=model, tools=[get_weather])
11 result = agent.invoke({"messages": [{"role": "user", "content": "서울 날씨 알려줘"}]})
12 print(result["messages"][-1].content)
```

서울은 현재 맑음 상태이며, 온도는 22도입니다.

Agent 설계 시 고려사항

상세 설명

- 도구 개수: 너무 많으면 LLM이 도구 선택에 혼란을 겪을 수 있음
- 명확한 도구 설명(docstring): LLM이 도구 용도를 이해하는 핵심 단서
- 최대 반복 횟수 제한: 무한 루프 방지
- 에러 처리: 도구 실행 실패 시 대응 로직 필요
- 비용 관리: 반복 호출이 많을수록 API 비용 증가

Agent 활용 사례

예제

- 여행 예약 에이전트: 항공권 검색 → 호텔 검색 → 예약 순차 처리
- 데이터 분석 에이전트: 질문 분석 → SQL 생성 → 실행 → 결과 요약
- 코드 수정 에이전트: 버그 위치 파악 → 코드 수정 → 테스트 실행
- 리서치 에이전트: 웹 검색 → 정보 취합 → 보고서 작성

Tools란 무엇인가?

핵심 개념

- Tools: LLM이 스스로 할 수 없는 작업(계산, 검색, DB 조회 등)을 수행하도록 돕는 함수
- 일반 Python 함수에 설명(docstring)을 추가해 도구로 등록
- LLM은 도구의 이름과 설명을 보고 호출 여부와 인자를 스스로 결정
- 도구 실행은 개발자의 코드에서 이루어지며, 결과만 LLM에 전달됨
- LangChain은 `@tool` 데코레이터로 손쉽게 도구를 정의하도록 지원

Tool 정의 방법

상세 설명

- 함수의 타입 힌트와 docstring이 LLM에게 전달되는 설명 역할
- 명확하고 구체적인 설명일수록 LLM이 올바르게 호출
- 여러 개의 도구를 리스트로 묶어 Agent/Model에 바인딩
- 도구는 검색, 계산, DB 조회, 외부 API 호출 등 다양한 형태 가능

```
1 from langchain_core.tools import tool
2
3 @tool
4 def add_numbers(a: int, b: int) -> int:
5     """두 정수를 더한 값을 반환합니다."""
6     return a + b
```

Tool 바인딩과 호출

상세 설명

- `model.bind_tools([...])` 로 모델에 도구 목록을 연결
- 모델은 사용자의 질문에 따라 도구 호출이 필요한지 판단
- 필요하다고 판단하면 응답에 `tool_calls` 정보 포함
- 개발자는 이 정보를 바탕으로 실제 함수를 실행하고 결과를 반환

```
1 model_with_tools = model.bind_tools([add_numbers])
2 response = model_with_tools.invoke("3과 5를 더하면?")
3 print(response.tool_calls)
```

```
[{'name': 'add_numbers', 'args': {'a': 3, 'b': 5}, 'id': 'call_DrXiNx5tIRfjQEzeeSovrmM5', 'type': 'tool_call'}]
```

Tools 실습: 검색 도구 연결

예제

```
1 from langchain_tavily import TavilySearch
2
3 search_tool = TavilySearch(max_results=3)
4
5 agent = create_agent(model=model, tools=[search_tool, add_numbers])
6 result = agent.invoke({
7     "messages": [{"role": "user", "content": "오늘 서울 날씨와 12+30을 알려줘"}]
8 })
9 print(result["messages"][-1].content)
```

오늘 서울의 날씨는 다음과 같습니다:

- 날씨: 흐리고 한때 비
- 최저/최고기온: 23°C / 27°C
- 강수확률: 60%
- 미세먼지/초미세먼지: 수신 데이터 없음

12와 30을 더하면 42가 나옵니다.

Tool 설계 Best Practice

상세 설명

- 하나의 도구는 하나의 명확한 기능만 수행하도록 설계
- 입력 인자는 최소한으로, 타입은 명확하게 지정
- 실패 가능성이 있는 도구는 에러 메시지를 반환하도록 처리
- 민감한 작업(결제, 삭제 등)은 사람의 확인(Human-in-the-loop) 단계 추가 고려

Integrations

- Integration 관점에서 바라 본 chat model
- 기타 컴포넌트

Integration 관점에서 바라 본 chat model

LLM 기반 애플리케이션 개발을 위한 LangChain 기본 컴포넌트 중 가장 기본이 되는 model component

```
init_chat_model(  
    model: str | None = None,  
    *,  
    model_provider: str | None = None,  
    configurable_fields: Literal["any"] | list[str] | tuple[str, ...] | None = None,  
    config_prefix: str | None = None,  
    **kwargs: Any,  
) -> BaseChatModel | _ConfigurableModel
```

[init_chat_model](#) : model 제공자와 이름을 통해 초기화 하는 함수

model : model 이름

model_provider : model 제공자

configurable_fields : 설정 가능한 model parameter 지정

config_prefix : 설정 가능한 model parameter의 prefix 지정

**kwargs : model 생성 시 전달하는 인자

Integration 관점에서 바라 본 chat model

Parameters

Name	Type	Description
model	str None	Default: None Name of the model to use, with provider prefix — e.g., <code>'openai:gpt-5.5'</code>. A bare model name (e.g., <code>'claude-opus-4-7'</code>) is also accepted; we will attempt to infer the provider from the prefix using the mapping below. Inference is best-effort and not guaranteed, so prefer the prefixed form when possible. Prefer pinned model IDs over moving aliases (e.g., <code>'claude-haiku-4-5-20251001'</code> rather than <code>'claude-haiku-4-5'</code>) so behavior does not drift if the alias is repointed upstream. Inferred providers by prefix (case-insensitive): <code>gpt-...</code> <code>o1...</code> <code>o3...</code> -> <code>openai</code> <code>claude...</code> -> <code>anthropic</code> <code>amazon....</code> <code>anthropic....</code> <code>meta....</code> -> <code>bedrock</code> <code>gemini...</code> -> <code>google_vertexai</code> (default changes in next major; pass <code>model_provider</code> to lock in) <code>command...</code> -> <code>cohere</code> <code>accounts/fireworks...</code> -> <code>fireworks</code> <code>mistral...</code> <code>mixtral...</code> -> <code>mistralai</code> <code>deepseek...</code> -> <code>deepseek</code> <code>grok...</code> -> <code>xai</code> <code>sonar...</code> -> <code>perplexity</code> <code>solar...</code> -> <code>upstage</code> <code>chatgpt...</code> <code>text-davinci...</code> -> <code>openai</code> (legacy)

model_provider str | None

Default: None

Provider of the model, passed separately instead of as a prefix on `model`.

Equivalent to the prefix form — e.g., `model='claude-sonnet-4-5'`, `model_provider='anthropic'` behaves the same as `model='anthropic:claude-sonnet-4-5'`.

Prefer the prefix form on `model` for most usage. Reach for this kwarg when:

- The provider is dynamic (read from config or an env var) and you'd otherwise concatenate strings.
- You want `model` and `model_provider` to be independently swappable at runtime via `configurable_fields` (e.g., to route the same model name to a different host).

Supported values and the integration package each requires:

- `openai` -> `langchain-openai`
- `anthropic` -> `langchain-anthropic`
- `azure_openai` -> `langchain-openai`
- `azure_ai` -> `langchain-azure-ai`
- `google_vertexai` -> `langchain-google-vertexai`
- `google_genai` -> `langchain-google-genai`
- `anthropic_bedrock` -> `langchain-aws`
- `bedrock` -> `langchain-aws`
- `bedrock_converse` -> `langchain-aws`
- `cohere` -> `langchain-cohere`
- `fireworks` -> `langchain-fireworks`
- `together` -> `langchain-together`
- `mistralai` -> `langchain-mistralai`
- `huggingface` -> `langchain-huggingface`
- `groq` -> `langchain-groq`
- `ollama` -> `langchain-ollama`

Integration 관점에서 바라 본 chat model

init_chat_model 함수를 통해 공통적인 인터페이스 제공

```
gpt_4o_mini = init_chat_model(  
    "openai:gpt-4o-mini",  
)  
  
gemini = init_chat_model(  
    "google_genai:gemini-2.5-flash-lite",  
)
```

The diagram illustrates the structure of the `init_chat_model` function arguments. Two labels, `model_provider` and `model`, are positioned at the bottom. From `model_provider`, two arrows point upwards: one to the string `"openai:gpt-4o-mini"` in the first function call and one to the string `"google_genai:gemini-2.5-flash-lite"` in the second function call. From `model`, two arrows point upwards: one to the string `"gpt-4o-mini"` in the first function call and one to the string `"gemini-2.5-flash-lite"` in the second function call. This indicates that the function expects a combined string where the provider and model name are separated by a colon.

Integration 관점에서 바라 본 chat model

The screenshot shows the GitHub web interface for the file `base.py` in the `langchain/chat_models` directory. The file contains a dictionary of built-in providers and a function to call them.

Annotation 1: `init_chat_model` (points to line 56)

Annotation 2: `"google_genai": ("langchain_google_genai", "ChatGoogleGenerativeAI", _call),` (points to line 72)

Annotation 3: `def _call(cls: type[BaseChatModel], **kwargs: Any) -> BaseChatModel:` (points to line 51)

Annotation 4: `# TODO: replace with operator.call when lower bounding to Python 3.11` (points to line 52)

Annotation 5: `return cls(**kwargs)` (points to line 53)

Annotation 6: `"openai": ("langchain_openai", "ChatOpenAI", _call),` (points to line 91)

Annotation 7: `Model_provider, model 정보로부터 해당 model을 초기화하고 사용하는 흐름을 확인해보기` (points to the function definition and the providers dictionary)

Chat models 연동 개요

핵심 개념

- LangChain은 수십 개 이상의 Chat model 제공사를 표준 인터페이스로 지원
- OpenAI, Anthropic, Google, Cohere, 로컬 모델(Ollama) 등 연동 가능
- 각 제공사별 패키지(`langchain-openai` , `langchain-anthropic` 등) 설치 필요
- `init_chat_model` 을 사용하면 제공사 전환이 코드 한 줄로 가능
- 멀티모달(이미지, 음성) 입력을 지원하는 모델도 증가하는 추세

Chat models 연동 실습

예제

```
1 from langchain.chat_models import init_chat_model
2
3 gpt_4o_mini = init_chat_model("openai:gpt-4o-mini")
4 gemini = init_chat_model("google_genai:gemini-2.5-flash-lite")
5
6 for m in [gpt_4o_mini, gemini]:
7     print(m.invoke("오늘 기분이 어때?").content + "\n\n")
```

저는 감정을 느끼지 않지만, 당신의 기분을 듣고 싶어요! 오늘 기분이 어떨까요? 도움이 필요하시면 말씀해 주세요.

저는 인공지능이기 때문에 감정을 느끼지 못합니다. 하지만 당신의 기분을 묻는 질문에 답할 수 있는 정보를 처리하고
당신은 오늘 기분이 어떠신가요? 당신의 기분을 알려주시면 제가 도움을 드릴 수 있는지 살펴보겠습니다.

Chat models 선택 기준

상세 설명

기준	설명
성능	추론/코드/멀티모달 능력 수준
비용	토큰당 과금 정책
지연시간	응답 속도
데이터 정책	데이터 학습 활용 여부, 보안 정책

- 서비스 목적과 예산에 맞는 모델 선택이 중요

Chat models 통합 관리

상세 설명

- 여러 모델을 동시에 활용하는 멀티모델 전략도 가능
- 간단한 작업은 저렴한 모델, 복잡한 작업은 고성능 모델로 라우팅
- Fallback 설정으로 특정 모델 장애 시 다른 모델로 자동 전환

```
model_with_fallback = model_primary.with_fallbacks([model_backup])
```

기타 컴포넌트

- Tools and toolkits: 다양한 도구 묶음 연동
- Middleware, Sandboxes, Backends: 실행 환경과 흐름 제어
- Checkpointers: 상태 저장 및 재개
- Retrievers, Text splitters, Embedding models, Vector stores: 검색 기반 응답(RAG)의 핵심 구성요소
- Document loaders: 다양한 문서 형식을 불러오는 방법

Tools and toolkits 개요

핵심 개념

- Toolkit은 관련된 여러 도구를 하나의 묶음으로 제공
- 예: SQL Database Toolkit, 파일 시스템 Toolkit, Gmail Toolkit
- 각 Toolkit은 해당 도메인에 필요한 도구들을 미리 구성해 제공
- 개발자가 도구를 하나하나 정의할 필요 없이 빠르게 활용 가능

대표 Toolkit 소개

상세 설명

Toolkit	제공 기능
SQLDatabaseToolkit	테이블 조회, 쿼리 실행
FileManagementToolkit	파일 읽기/쓰기/목록 조회
RequestsToolkit	HTTP 요청 수행
GmailToolkit	메일 조회/발송

- 필요한 Toolkit을 설치하고 Agent에 바로 연결 가능

Toolkit 실습

예제

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///company.db")
toolkit = SQLDatabaseToolkit(db=db, llm=model)

agent = create_agent(model=model, tools=toolkit.get_tools())
result = agent.invoke({"messages": [{"role": "user", "content": "직원 수를 알려줘"}]})
print(result["messages"][-1].content)
```

커스텀 Toolkit 구성

상세 설명

- 여러 개의 `@tool` 함수를 리스트로 묶으면 간단한 커스텀 Toolkit 구성 가능
- 사내 시스템(ERP, CRM 등) 전용 Toolkit을 직접 개발하는 경우도 많음
- 도구 간 일관된 네이밍 규칙을 정하면 유지보수에 유리
- 보안이 중요한 도구는 권한 검증 로직을 반드시 포함

Toolkit 설계 시 유의사항

상세 설명

- 도구 수가 많아지면 LLM의 선택 정확도가 떨어질 수 있음
- 유사한 기능의 도구는 하나로 통합하는 것이 유리
- 각 도구의 설명은 중복 없이 명확하게 작성
- 프로덕션 배포 전 충분한 테스트로 오작동 방지

Middleware 개요 (Integration 관점)

핵심 개념

- Middleware: Agent 실행 파이프라인의 전/후 단계에 개입하는 컴포넌트
- 로깅, 입력 검증, 응답 필터링 등 공통 로직을 재사용 가능하게 함
- 여러 Middleware를 순서대로 연결(Chain)해 파이프라인 구성

Middleware의 활용 예

상세 설명

- 로깅 Middleware: 모든 요청/응답을 기록
- 속도 제한(Rate Limiting) Middleware: 과도한 호출 방지
- 콘텐츠 필터 Middleware: 부적절한 입력/출력 차단
- 인증 Middleware: 사용자 권한 확인 후 실행 허용

Middleware 연동 예제

예제

```
def logging middleware(state, next):  
    print(f"요청 시작: {state['messages'][-1].content}")  
    result = next(state)  
    print("요청 완료")  
    return result
```

Sandboxes란 무엇인가?

핵심 개념

- Sandbox: 코드나 도구를 격리된 안전한 환경에서 실행하는 공간
- 특히 LLM이 생성한 코드를 직접 실행할 때 보안상 매우 중요
- 시스템 파일 접근, 네트워크 사용 등을 제한적으로 허용
- 대표적으로 Docker 컨테이너, 클라우드 기반 실행 환경 등이 활용됨

Sandbox가 필요한 이유

상세 설명

- LLM이 생성한 코드에는 의도치 않은 오류나 악성 코드가 포함될 수 있음
- 실제 서버 환경에서 직접 실행하면 시스템 전체에 영향 가능
- Sandbox는 문제가 발생해도 본 시스템에 영향을 주지 않도록 격리
- 코드 실행형 에이전트(Code Interpreter 등)의 핵심 안전장치

Sandbox 활용 실습 개념

예제

```
# 개념적 예시: 코드 실행 도구를 Sandbox 환경에서 호출
result = sandbox tool.run(code="print(1 + 1)")
print(result.output) # "2"
```

Sandbox 적용 시 고려사항

상세 설명

- 실행 시간 제한(Timeout)으로 무한 루프 방지
- 메모리/CPU 사용량 제한 설정 필요
- 외부 네트워크 접근 여부를 명확히 정책화
- 실행 결과 로그를 남겨 사후 감사(Audit) 가능하게 구성

Sandboxes 정리

핵심 요약

- Sandbox는 도구/코드 실행을 안전하게 격리하는 환경
- 코드 생성·실행형 에이전트에서 특히 중요
- 리소스 제한과 로깅으로 안전성을 높일 수 있음

Backends란 무엇인가?

핵심 개념

- Backend: Agent/Chain이 실제로 실행되고 상태가 관리되는 인프라 계층
- 로컬 메모리 기반부터 분산 서버 기반까지 다양하게 구성 가능
- 어떤 Backend를 쓰느냐에 따라 확장성, 안정성이 달라짐
- Checkpointer, 캐시 저장소 등도 넓은 의미의 Backend에 포함

Backend 유형 비교

상세 설명

Backend 유형	특징
In-Memory	빠르지만 서버 재시작 시 데이터 소실
SQLite/Postgres	영구 저장, 중소규모 서비스에 적합
Redis	빠른 캐시성 저장, 세션 관리에 유리
분산 클라우드 인프라	대규모 트래픽 처리에 적합

Backend 선택 실습 예제

예제

```
from langgraph.checkpoint.postgres import PostgresSaver

checkpointer = PostgresSaver.from_conn_string(
    "postgresql://user:pass@localhost:5432/langgraph"
)
agent = create_agent(model=model, tools=[], checkpointer=checkpointer)
```

Checkpointers란 무엇인가?

핵심 개념

- Checkpointer: LangGraph에서 실행 상태를 특정 시점 기준으로 저장하는 컴포넌트
- Short-term memory 구현의 핵심 기반 기술
- 각 단계(Step)마다 상태를 저장해 중단 후 재개가 가능
- Thread ID로 여러 세션의 상태를 독립적으로 관리

Checkpoint의 종류

상세 설명

Checkpoint	특징
InMemorySaver	개발/테스트용, 휘발성
SqliteSaver	파일 기반 영구 저장
PostgresSaver	프로덕션급 영구 저장

- 운영 환경에서는 영구 저장이 가능한 Checkpointer 사용 권장

Checkpointer 실습

예제

```
from langgraph.checkpoint.sqlite import SqliteSaver

checkpointer = SqliteSaver.from_conn_string("checkpoints.db")
agent = create_agent(model=model, tools=[], checkpointer=checkpointer)

config = {"configurable": {"thread_id": "student-01"}}
agent.invoke({"messages": [{"role": "user", "content": "복습 시작할게"}]}, config)
```

Checkpointter 활용: 재개와 롤백

상세 설명

- 특정 시점(Checkpoint)으로 되돌아가는 기능(Time Travel) 지원
- 오류 발생 시 이전 정상 상태로 복구 가능
- 사람이 중간에 개입(Human-in-the-loop)하는 시나리오에 필수적
- 디버깅 시 특정 단계의 상태를 그대로 재현 가능

Checkpointing 활용 시 성능 고려

상세 설명

- 매 단계 저장이므로 저장소 I/O 비용 발생
- 상태 데이터가 커지면 저장/조회 속도에 영향
- 오래된 세션은 주기적으로 정리(삭제) 정책 필요
- 저장소 선택은 서비스 트래픽 규모에 맞게 결정

Retrievers란 무엇인가?

핵심 개념

- Retriever: 질문과 관련된 정보를 데이터 소스에서 검색해 가져오는 컴포넌트
- RAG(검색 증강 생성)의 핵심 구성요소
- 벡터 검색, 키워드 검색 등 다양한 검색 방식을 표준 인터페이스로 제공
- Vector Store를 감싸서 `retriever.invoke(query)` 형태로 손쉽게 사용

Retriever의 동작 원리

상세 설명

- 사용자 질문을 임베딩(벡터)으로 변환
- Vector Store에서 가장 유사한 문서 조각들을 검색
- 검색된 문서를 LLM에게 함께 전달해 답변 생성에 활용
- 이 과정을 통해 LLM이 모르는 최신/사내 정보도 답변 가능

Retriever 실습

예제

```
retriever = vector_store.as_retriever(search_kwargs={"k": 3})
docs = retriever.invoke("환불 정책이 어떻게 되나요?")

for d in docs:
    print(d.page_content[:100])
```

Retriever 종류

상세 설명

Retriever 유형	설명
Similarity Retriever	벡터 유사도 기반 검색
MMR Retriever	유사도 + 다양성을 함께 고려
Self-query Retriever	질문에서 필터 조건까지 자동 추출
Multi-query Retriever	하나의 질문을 여러 방식으로 변형해 검색

Retriever 성능 향상 팁

상세 설명

- 검색 결과 수(k)를 적절히 조정해 관련성과 비용의 균형 유지
- 문서를 잘게 나누는 Text splitter 전략이 검색 품질에 큰 영향
- 필요 시 Reranking(재정렬) 기법으로 상위 결과 품질 향상
- 도메인 특화 데이터는 전용 임베딩 모델 사용 고려

Retrievers 정리

핵심 요약

- Retriever는 질문과 관련된 문서를 검색해오는 RAG의 핵심 요소
- 벡터 유사도 기반 검색이 가장 대표적인 방식
- MMR, Self-query 등 다양한 고급 검색 전략 존재

Text splitters란 무엇인가?

핵심 개념

- Text splitter: 긴 문서를 검색에 적합한 크기의 작은 조각(Chunk)으로 나누는 도구
- Context Window와 검색 정확도를 고려해 적절한 크기로 분할
- 문장이 중간에 잘리지 않도록 의미 단위를 고려한 분할이 중요
- RAG 파이프라인에서 문서 로딩 직후 수행되는 필수 단계

Text splitter의 종류

상세 설명

Splitter	설명
CharacterTextSplitter	지정 문자 수 기준으로 단순 분할
RecursiveCharacterTextSplitter	문단→문장→단어 순으로 재귀적 분할
TokenTextSplitter	토큰 수 기준으로 분할
MarkdownHeaderTextSplitter	마크다운 제목 구조 기준으로 분할

- 문서 특성에 맞는 Splitter 선택이 중요

Text splitter 실습

예제

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_text(long_document_text)
print(len(chunks), "개의 조각으로 분할됨")
```

Chunk 크기 설계 전략

상세 설명

- Chunk가 너무 작으면: 문맥이 부족해 검색 품질 저하
- Chunk가 너무 크면: 불필요한 정보까지 포함되어 정확도 저하
- Overlap(겹침)을 두면 경계에서 정보 손실 방지
- 일반적으로 300~1000자 사이에서 실험적으로 최적값 탐색

Embedding models란 무엇인가?

핵심 개념

- Embedding: 텍스트를 숫자 벡터로 변환하는 기술
- 의미가 비슷한 텍스트는 벡터 공간에서 가까운 위치에 배치됨
- 이 벡터 간 거리를 계산해 유사도 검색이 가능해짐
- OpenAI, Cohere, HuggingFace 등 다양한 Embedding 모델 제공사 존재

Embedding의 동작 원리

상세 설명

- 문장을 입력하면 수백~수천 차원의 실수 벡터로 출력
- 코사인 유사도(Cosine Similarity) 등으로 벡터 간 유사도 계산
- 같은 주제의 문장일수록 벡터 방향이 유사하게 학습됨
- 임베딩 모델은 검색뿐 아니라 분류, 클러스터링에도 활용

Embedding 실습

예제

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector = embeddings.embed_query("LangChain은 무엇인가요?")

print(len(vector))      # 벡터 차원 수
print(vector[:5])       # 앞부분 값 확인
```

Embedding 모델 선택 기준

상세 설명

기준	설명
차원 수	클수록 표현력↑, 저장/계산 비용↑
언어 지원	다국어(한국어 포함) 지원 여부
비용	임베딩 생성 비용
도메인 특화	특정 분야 전용 모델 존재 여부

Vector stores란 무엇인가?

핵심 개념

- Vector store: 임베딩된 벡터를 저장하고 빠르게 검색할 수 있게 하는 데이터베이스
- 대표적인 Vector store: Chroma, FAISS, Pinecone, Weaviate
- 저장된 벡터 중 질문 벡터와 가장 유사한 것들을 빠르게 찾아줌
- RAG 파이프라인의 핵심 저장소 역할

Vector store 종류 비교

상세 설명

Vector Store	특징
Chroma	가볍고 로컬 실행에 적합
FAISS	빠른 유사도 검색, Meta 개발
Pinecone	완전관리형 클라우드 서비스
Weaviate	하이브리드 검색(키워드+벡터) 지원

- 프로토타입은 Chroma/FAISS, 프로덕션은 관리형 서비스 고려

Vector store 구축 실습

예제

```
from langchain chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = Chroma.from_texts(
    texts=chunks,
    embedding=embeddings
)

results = vector_store.similarity_search("환불 규정 알려줘", k=2)
for r in results:
    print(r.page_content)
```

Vector store 운영 고려사항

상세 설명

- 데이터가 자주 바뀌는 경우 주기적 재색인(Re-indexing) 필요
- 메타데이터(출처, 날짜 등)를 함께 저장해 필터링 검색 지원
- 검색 속도와 정확도 간 트레이드오프 존재 (인덱스 알고리즘 선택)
- 대규모 서비스는 클라우드 관리형 Vector store 고려

Vector store와 RAG 파이프라인

상세 설명

- 문서 로딩 → 분할(Splitter) → 임베딩 → Vector store 저장 → 검색(Retriever) 순으로 진행
- 이 전체 흐름을 RAG(Retrieval-Augmented Generation)이라 부름
- Vector store는 이 파이프라인의 중심 저장소 역할
- 질문마다 관련 문서를 찾아 LLM에게 제공해 정확도 향상

Document loaders란 무엇인가?

핵심 개념

- Document loader: PDF, Word, 웹페이지 등 다양한 형식의 문서를 표준화된 형태로 불러오는 도구
- 로드된 문서는 Document 객체(내용 + 메타데이터)로 변환됨
- RAG 파이프라인의 가장 첫 단계
- 파일뿐 아니라 데이터베이스, API 결과도 로드 가능

다양한 Document loader

상세 설명

Loader	지원 형식
PyPDFLoader	PDF 문서
Docx2txtLoader	Word 문서
WebBaseLoader	웹페이지
CSVLoader	CSV 데이터
NotionDirectoryLoader	Notion 페이지

Document loader 실습

예제

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("company_policy.pdf")
documents = loader.load()

print(len(documents), "페이지 로드됨")
print(documents[0].page_content[:200])
```

Document loader와 메타데이터

상세 설명

- 각 Document 는 page_content (본문)와 metadata (출처, 페이지 번호 등)로 구성
- 메타데이터는 이후 검색 필터링과 출처 표시에 활용
- 여러 문서를 로드할 때 출처를 명확히 남겨두는 것이 중요
- 실무에서는 사내 다양한 문서 포맷을 통합 로딩하는 파이프라인 구축

Runnable의 이해

- LCEL – Runnable 인터페이스
- 주요 Runnable 파생 클래스 및 래퍼
- 주요 컴포넌트의 RunnableSerializable 구현
- RunnableSequence, RunnableParallel

LCEL – Runnable 인터페이스

LCEL(LangChain Expression Language)의 핵심 기반 프로토콜이자 추상 인터페이스로, 서로 다른 구성 요소(프롬프트, 모델, 출력 파서 등)를 일관된 방식으로 연결하고 실행할 수 있도록 표준화

- Runnable의 핵심 개념 및 설계 목적
 - 표준화된 인터페이스 제공 : 모든 컴포넌트(PromptTemplate, ChatModel, OutputParser, Retriever 등)가 동일한 실행 메서드를 공유하도록 규격화
 - 선언적 파이프라인 구성 : 리눅스 파이프 연산자(|)를 활용해 컴포넌트 간 입출력을 직관적으로 연결 (chain = prompt | model | parser)

메서드 형태	동기(Sync)	비동기(Async)	주요 용도
단일 호출	invoke(input, config)	ainvoke(input, config)	단일 입력값 처리 및 최종 결과 반환
배치 호출	batch(inputs, config)	abatch(inputs, config)	여러 입력값을 병렬/동시 처리하여 성능 최적화
스트리밍	stream(input, config)	astream(input, config)	LLM의 토큰 생성 등 결과를 실시간 청크 단위로 스트리밍

Pipe(|) 연산자란?

핵심 개념

- Pipe 연산자(|): 앞 단계의 출력을 다음 단계의 입력으로 자동 전달
- Unix 파이프(|)와 유사한 개념으로 데이터 흐름을 직관적으로 표현
- `prompt | model | parser` 형태로 전체 파이프라인을 한 줄로 표현 가능
- 각 단계는 독립적으로 테스트하고 재사용할 수 있음

```
chain = prompt | model | output_parser
```

Pipe 연산자 기본 실습

예제

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_core.output_parsers import StrOutputParser
3
4 prompt = ChatPromptTemplate.from_template("{topic}에 대해 한 문장으로 설명해줘")
5 parser = StrOutputParser()
6
7 chain = prompt | model | parser
8 result = chain.invoke({"topic": "머신러닝"})
9 print(result)
10
```

머신러닝은 컴퓨터가 명시적으로 프로그래밍 되지 않아도 스스로 학습하고 더 나은 성능을 내기 위해 데이터를 이해하

Pipe 연산자의 장점

상세 설명

- 코드가 간결하고 가독성이 높음
- 각 단계를 독립적인 함수처럼 재사용 가능
- 새로운 단계를 추가/제거하기 쉬운 유연한 구조
- 디버깅 시 특정 단계만 따로 실행해 확인 가능

```
1 debug_chain = prompt | model
2 print(debug_chain.invoke({"topic": "딥러닝"}))
3
```

```
content='딥러닝은 인공 신경망을 이용해 데이터를 학습하고, 예측하거나 분류하는 인공지능의 한 분야입니다.' additi
```

Pipe 체인에 함수 추가하기

상세 설명

- 일반 Python 함수도 RunnableLambda 로 감싸 체인에 포함 가능
- 전처리, 후처리 로직을 체인 안에 자연스럽게 통합

```
1 from langchain_core.runnables import RunnableLambda
2
3 def to_upper(text: str) -> str:
4     return text.upper()
5
6 prompt = ChatPromptTemplate.from_template("Explain about the {topic} for elementary school
7
8 chain = prompt | model | parser | RunnableLambda(to_upper)
9 result = chain.invoke({"topic": "Machine Learning"})
10
11 print(result)
```

MACHINE LEARNING IS LIKE TEACHING A COMPUTER TO PLAY A GAME. LET'S SAY YOU WANT TO PLAY A GAME OF CHESS, BUT YOU
SIMILARLY, IN MACHINE LEARNING, WE ARE TEACHING THE COMPUTER TO UNDERSTAND PATTERNS AND LEARN FROM IT, JUST LIKE
JUST LIKE THE WAY YOU BECOME BETTER AT PLAYING CHESS AS YOU PLAY MORE AND MORE, THE COMPUTER ALSO GETS BETTER AT

주요 Runnable 파생 클래스 및 래퍼

- RunnablePassthrough
 - 입력을 변형하지 않고 그대로 다음 단계로 전달
 - assign() 메서드를 통해 기존 입력을 유지하면서 새로운 키-값 쌍을 병합할 때 주로 사용
- RunnableParallel (또는 딕셔너리 리터럴 {})
 - 여러 Runnable을 병렬로 동시 실행하고, 그 결과를 딕셔너리 형태로 취합
 - 예: 검색기(Retriever) 결과와 원본 질문을 동시에 프롬프트에 매핑할 때 활용
- RunnableLambda
 - 일반 파이썬 함수(def 또는 lambda)를 Runnable 인터페이스로 변환
 - 체인中间的 커스텀 데이터 전처리/후처리 작업에 필수적
- RunnableBranch
 - 조건식에 따라 서로 다른 Runnable 체인으로 분기 처리 (if-else 패턴)
- RunnableWithMessageHistory
 - 상태가 없는 체인에 세션별 대화 기록(Memory)을 자동으로 주입하고 저장하도록 래핑

PromptTemplate – RunnableSerializable 구현

```
37
38  ✓ class BasePromptTemplate(
39      RunnableSerializable[dict[str, Any], PromptValue], ABC, Generic[FormatOutputType]
40  ):
41      """Base class for all prompt templates, returning a prompt."""
42
327
328  ✓ class StringPromptTemplate(BasePromptTemplate[str], ABC):
329      """String prompt that exposes the format method, returning a prompt."""
330
23
24  ✓ class PromptTemplate(StringPromptTemplate):
25      """Prompt template for a language model.
```

```
graph TD
    BasePromptTemplate --> StringPromptTemplate
    StringPromptTemplate --> PromptTemplate
```

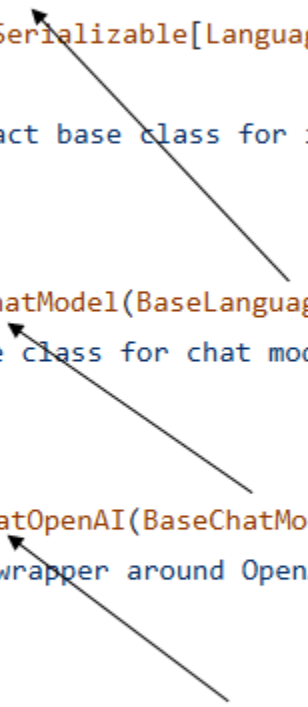
StrOutputParser – RunnableSerializable 구현

```
139
140  class BaseOutputParser(
141      BaseLLMOutputParser[T], RunnableSerializable[LanguageModelOutput, T]
142  ):
143      """Base class to parse the output of an LLM call.
144
27
28  class BaseTransformOutputParser(BaseOutputParser[T]):
29      """Base class for an output parser that can handle streaming input."""
30
7
8  class StrOutputParser(BaseTransformOutputParser[str]):
9      """Extract text content from model outputs as a string.
10
```

```
graph TD
    StrOutputParser --> BaseTransformOutputParser
    BaseTransformOutputParser --> BaseOutputParser
```

ChatOpenAI – RunnableSerializable 구현

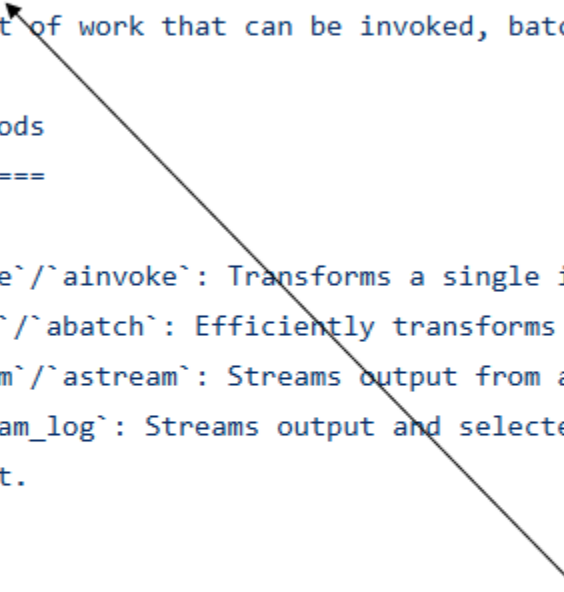
```
180
181  class BaseLanguageModel(
182      RunnableSerializable[LanguageModelInput, LanguageModelOutputVar], ABC
183  ):
184      """Abstract base class for interfacing with language models.
185
283
284  class BaseChatModel(BaseLanguageModel[AIMessage], ABC):
285      r"""Base class for chat models.
286
711
712  class BaseChatOpenAI(BaseChatModel):
713      """Base wrapper around OpenAI large language models for chat.
714
2797
2798  class ChatOpenAI(BaseChatOpenAI): # type: ignore[override]
2799      r"""Interface to OpenAI chat model APIs.
2800
```



```
graph BT
    BaseChatOpenAI --> BaseChatModel
    BaseChatModel --> BaseLanguageModel
```

RunnableSerializable – Runnable 구현

```
132
133  ✓ class Runnable(ABC, Generic[Input, Output]):
134      """A unit of work that can be invoked, batched, streamed, transformed and composed.
135
136      Key Methods
137      =====
138
139      - `invoke`/`ainvoke`: Transforms a single input into an output.
140      - `batch`/`abatch`: Efficiently transforms multiple inputs into outputs.
141      - `stream`/`astream`: Streams output from a single input as it's produced.
142      - `astream_log`: Streams output and selected intermediate results from an
143          input.
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287  ✓ class RunnableSerializable(Serializable, Runnable[Input, Output]):
288      """Runnable that can be serialized to JSON."""
289
```



RunnableSequence란 무엇인가?

핵심 개념

- RunnableSequence: 여러 Runnable을 순서대로 실행하는 체인 객체
- 사실 `A | B | C` 코드는 내부적으로 `RunnableSequence(A, B, C)` 로 변환됨
- 각 단계의 출력이 다음 단계의 입력으로 그대로 전달됨
- Pipe 연산자는 RunnableSequence를 만드는 문법적 편의 기능(Syntactic Sugar)

RunnableSequence 명시적 생성

상세 설명

- Pipe 연산자 대신 RunnableSequence 를 직접 생성할 수도 있음
- 동적으로 체인을 구성해야 할 때 유용

```
1 from langchain_core.runnables import RunnableSequence
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.output_parsers import StrOutputParser
4
5 prompt = ChatPromptTemplate.from_template("{topic}에 대해 한 문장으로 설명해줘")
6 parser = StrOutputParser()
7
8 #chain = prompt | model | parser
9 chain = RunnableSequence(prompt, model, parser)
10 result = chain.invoke({"topic": "머신러닝"})
11
12 print(result)
```

머신러닝은 컴퓨터가 데이터를 학습하고, 이를 통해 스스로 예측하거나 판단하는 능력을 개발하는 인공지능의 한 분야임

RunnableSequence의 데이터 흐름

상세 설명

- 입력 → Prompt(입력을 메시지로 변환) → Model(메시지로 응답 생성) → Parser(응답 후처리) → 출력
- 각 단계는 입력 타입과 출력 타입이 명확히 정의되어 있어야 함
- 타입이 맞지 않으면 실행 시점에 오류 발생
- 디버깅 시 각 단계의 입출력을 개별적으로 확인하는 것이 효과적

RunnableSequence 실습: 다단계 체인

예제

```
1 summarize_prompt = ChatPromptTemplate.from_template("다음 글을 요약해줘: {text}")
2 translate_prompt = ChatPromptTemplate.from_template("다음을 영어로 번역해줘: {summary}")
3
4 summarize_chain = summarize_prompt | model | parser
5 translate_chain = translate_prompt | model | parser
6
7 full_chain = (
8     {"summary": summarize_chain}
9     | translate_chain
10 )
11 result = full_chain.invoke({"text": """"미국에서 50명 이하의 소규모 하객만 초대하는 '마이크로 웨딩'이 새로운 결혼식 트렌드로 !
12 최근 아시아경제가 인용한 미국 매체 코스모폴리탄은 마이크로 웨딩이 새로운 결혼식 트렌드로 !
13 마이크로 웨딩은 명확한 기준은 없으나 통상 하객 50명 미만의 결혼식을 뜻한다. 미국 결혼 정보
14 print(result)
```

In the U.S., 'micro weddings'—inviting fewer than 50 guests—are becoming popular, reflecting rising living cost

In the U.S., 'micro weddings'—inviting fewer than 50 guests—are becoming popular, reflecting rising living costs and changing values among young generations. This indicates a retreat from traditional large-scale weddings and represents a wedding culture that emphasizes meaningful communication. Last year, the average number of guests at American weddings was 117, showing that micro weddings constitute a size reduction of more than half.

RunnableSequence 오류 처리

상세 설명

- 특정 단계에서 오류 발생 시 `.with_retry()` 로 재시도 설정 가능
- `.with_fallbacks()` 로 오류 시 대체 체인 실행 가능
- 각 단계별 실행 시간을 측정해 병목 구간 파악 가능

```
safe_chain = chain.with_retry(stop_after_attempt=3)
```

RunnableParallel이란 무엇인가?

핵심 개념

- RunnableParallel: 여러 Runnable을 동시에(병렬로) 실행하는 구조
- 디렉터리 형태로 여러 체인을 정의하면 자동으로 병렬 실행됨
- 서로 독립적인 여러 작업을 동시에 처리해 속도 향상
- 예: 하나의 질문에 대해 요약과 번역을 동시에 수행

RunnableParallel 실습

예제

```
1 from langchain_core.runnables import RunnableParallel
2
3 summarize_prompt = ChatPromptTemplate.from_template("다음 글을 요약해줘: {text}")
4 keyword_prompt = ChatPromptTemplate.from_template("다음 글에서 핵심 키워드를 추출해줘: {text}")
5
6 parallel_chain = RunnableParallel(
7     summary=summarize_prompt | model | parser,
8     keywords=keyword_prompt | model | parser
9 )
10
11 result = parallel_chain.invoke({"text": "미국에서 50명 이하의 소규모 하객만 초대하는 '마이크로 웨딩'이 유행하고 있다. 이는 전통적인 대규모 결혼식 대신 의미 있는 소통을 중시하는 결혼 문화의 변화를 반영한 것으로, 이를 보고한 미국 매체 코스모폴리탄은 결혼식 계획에서 가장 중요한 것이 하객 수를 결정하는 것이라고 전했다."})
12 print(result)
13 print(result["summary"])
14 print(result["keywords"])
```

{'summary': '미국에서는 젊은 세대의 변화된 가치관과 물가 상승을 고려해 50명 미만의 소규모 하객을 부르는 '마이크로 웨딩'이 유행하고 있다. 이는 전통적인 대규모 결혼식 대신 의미 있는 소통을 중시하는 결혼 문화의 변화를 반영한 것으로, 이를 보고한 미국 매체 코스모폴리탄은 결혼식 계획에서 가장 중요한 것이 하객 수를 결정하는 것이라고 전했다.', 'keywords': '미국, 50명, 소규모 하객, '마이크로 웨딩', 트렌드, 확산, 물가, 젊은 세대, 가치관, 대규모 결혼식, 소통, 결혼 문화'}

{'summary': '미국에서는 젊은 세대의 변화된 가치관과 물가 상승을 고려해 50명 미만의 소규모 하객을 부르는 '마이크로 웨딩'이 유행하고 있다. 이는 전통적인 대규모 결혼식 대신 의미 있는 소통을 중시하는 결혼 문화의 변화를 반영한 것으로, 이를 보고한 미국 매체 코스모폴리탄은 결혼식 계획에서 가장 중요한 것이 하객 수를 결정하는 것이라고 전했다.', 'keywords': '미국, 50명, 소규모 하객, '마이크로 웨딩', 트렌드, 확산, 물가, 젊은 세대, 가치관, 대규모 결혼식, 소통, 결혼 문화, 아시아경제, 코스모폴리탄, 결혼식 계획, 변화, 하객 50명 미만의 결혼식, 더 노트, 자료, 결혼식 평균 하객 수, 117명.'}

미국에서는 젊은 세대의 변화된 가치관과 물가 상승을 고려해 50명 미만의 소규모 하객을 부르는 '마이크로 웨딩'이 유행하고 있다. 이는 전통적인 대규모 결혼식 대신 의미 있는 소통을 중시하는 결혼 문화의 변화를 반영한 것으로, 이를 보고한 미국 매체 코스모폴리탄은 결혼식 계획에서 가장 중요한 것이 하객 수를 결정하는 것이라고 전했다.

미국, 50명, 소규모 하객, '마이크로 웨딩', 트렌드, 확산, 물가, 젊은 세대, 가치관, 대규모 결혼식, 소통, 결혼 문화, 아시아경제, 코스모폴리탄, 결혼식 계획, 변화, 하객 50명 미만의 결혼식, 더 노트, 자료, 결혼식 평균 하객 수, 117명.

RunnableParallel의 성능 이점

상세 설명

- 각 작업이 서로 의존하지 않을 때 전체 처리 시간 단축
- 예: 순차 실행 시 5초+5초=10초 → 병렬 실행 시 약 5초
- 여러 개의 검색/모델 호출을 동시에 수행할 때 특히 유용
- 다만 병렬 실행이 많아지면 API Rate Limit에 유의해야 함

RunnableParallel과 RunnableSequence의 조합

상세 설명

- 실제 서비스에서는 병렬과 순차를 함께 조합하는 경우가 많음
- 예: 병렬로 여러 정보를 수집한 뒤, 순차적으로 최종 답변 생성

```
full chain = (  
    RunnableParallel(context=retriever, question=lambda x: x["question"])  
    | final_prompt  
    | model  
    | parser  
)
```

RunnableParallel 활용 사례

예제

- 다각도 분석: 감정 분석 + 키워드 추출 + 요약물 동시에 수행
- 다중 검색: 여러 Vector store에서 동시에 검색 후 결과 통합
- 다국어 서비스: 여러 언어로 동시에 번역하여 제공

AI 서비스 개발을 위한 UI

- Gradio

Gradio란 무엇인가?

핵심 개념

- Gradio: 머신러닝/AI 모델의 데모 UI를 빠르게 만드는 Python 라이브러리
- 함수 하나를 감싸는 것만으로 입력-출력 인터페이스 자동 생성
- Hugging Face와의 강력한 통합으로 모델 공유가 매우 쉬움
- 이미지, 오디오, 텍스트 등 다양한 입출력 타입 기본 지원

Gradio 기본 구조

상세 설명

```
1 import gradio as gr
2
3 def chatbot_response(message, history):
4     response = agent.invoke({"messages": [{"role": "user", "content": message}]})
5     return response["messages"][-1].content
6
7 demo = gr.ChatInterface(fn=chatbot_response)
8 demo.launch()
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires 'share=True'. Automatically setting 'share=True' (:

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

+ Running on public URL: <https://350e13214a23fe2073.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run 'gradio

Chatbot

Type a message...

Gradio 컴포넌트 활용

상세 설명

컴포넌트	용도
gr.Textbox	텍스트 입력/출력
gr.Image	이미지 입력/출력
gr.Audio	오디오 입력/출력
gr.Slider	숫자 파라미터 조절

- 여러 컴포넌트를 조합해 커스텀 인터페이스 구성 가능

Gradio Blocks로 커스텀 UI 만들기

예제

```
1 with gr.Blocks() as demo:
2     gr.Markdown("## LangChain 기반 문서 요약기")
3     input_text = gr.Textbox(label="원문 입력")
4     output_text = gr.Textbox(label="요약 결과")
5     btn = gr.Button("요약하기")
6     btn.click(fn=summarize_chain.invoke, inputs=input_text, outputs=output_text)
7
8 demo.launch()
9
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires 'share=True'. Automatically setting 'share=True' (y

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

+ Running on public URL: <https://0358075be1b0bec710.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run 'gradio

LangChain 기반 문서 요약기

원문 입력

요약 결과

요약하기